Understanding Neural Network Optimization through River-Valley Dynamics

Chulhee "Charlie" Yun
Associate Professor at KAIST Graduate School of AI



KAIST AI
Kim Jaechul Graduate School

Optimization in Deep Learning

"Training" a neural network $f(\,\cdot\,;\Theta)$ given a finite dataset $\{(x_i, y_i)\}_{i=1}^n$

$$\text{minimize}_\Theta \quad R(\Theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(f(x_i;\Theta), y_i)$$

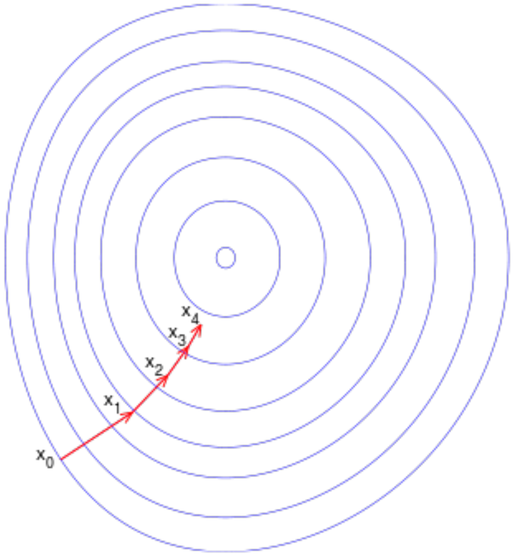
Use optimizers to minimize the training loss R by tuning parameters Θ

A high-dimensional & nonconvex optimization problem, but solved reliably!

Gradient Descent & Stochastic Gradient Descent

A fundamental algorithm for minimization: Gradient Descent (GD)

$$\Theta_{t+1} \leftarrow \Theta_t - \eta \nabla R(\Theta_t).$$



Idea: compute the gradient of R at current iterate Θ_t,

make a small step toward negative gradient.

η is called step size or learning rate.

In practice, we use Stochastic Gradient Descent (SGD)

$$\Theta_{t+1} = \Theta_t - \eta g_t$$

where g_t is the noisy gradient calculated with a subset of data

Image from https://en.wikipedia.org/wiki/Gradient_descent

Adam & AdamW

Adaptive learning rate methods: adjust coordinate-wise LRs for faster progress

Idea: Estimate the first and second moments of gradients

If past gradient values have been "small", increase LR, and vice versa

Adam

$$\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1) g_t \quad \text{(aka "momentum")}$$

$$\boldsymbol{v}_t = \beta_2 \boldsymbol{v}_{t-1} + (1-\beta_2) g_t^{\odot 2}$$

$$\Theta_{t+1} = \Theta_t - \eta \frac{\boldsymbol{m}_t}{\sqrt{\boldsymbol{v}_t} + \epsilon}$$

AdamW

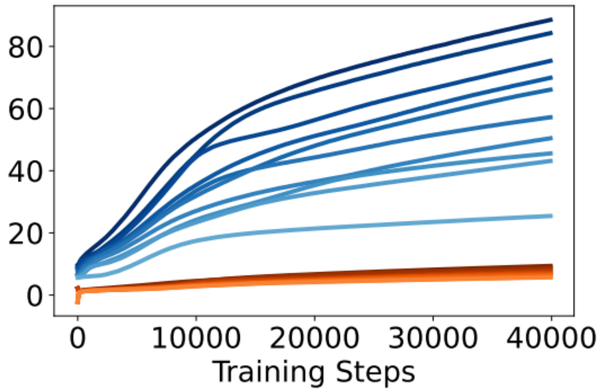
$$\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1) g_t$$

$$\boldsymbol{v}_t = \beta_2 \boldsymbol{v}_{t-1} + (1-\beta_2) g_t^{\odot 2}$$

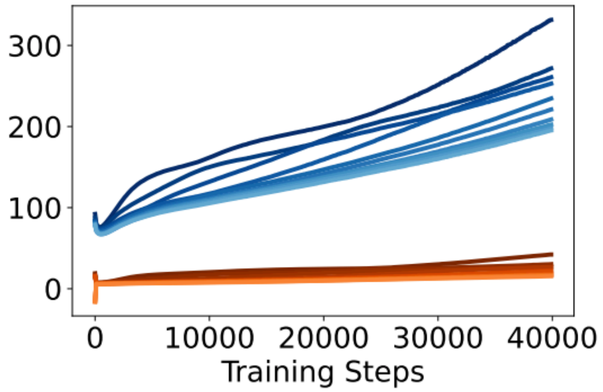
$$\Theta_{t+1} = \Theta_t - \eta \frac{\boldsymbol{m}_t}{\sqrt{\boldsymbol{v}_t} + \epsilon} - \eta\lambda\Theta_t$$

SGD Happens in a Tiny Subspace?

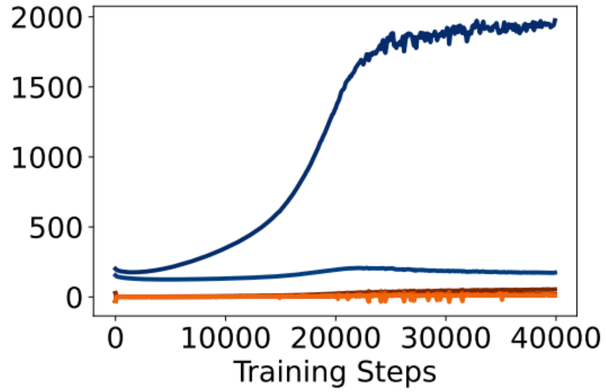
It has long been observed that for k-class classification, the top-k eigenvalues of the training loss Hessian $\nabla^2 R(\Theta_t)$ stand out as outliers



(a) MLP on MNIST-5k



(b) CNN on CIFAR10-5k

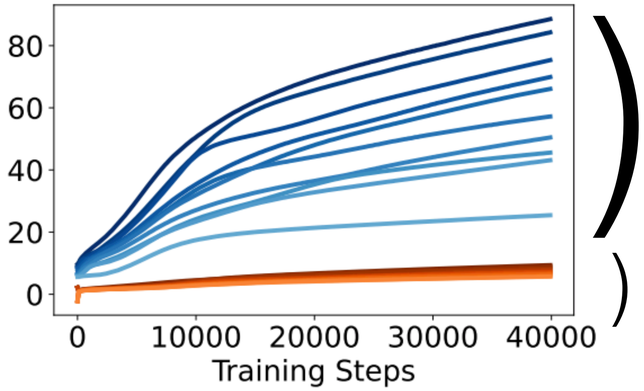


(c) Transformer on SST2-1k

[Sagun et al. (2016;2017); Ghorbani et al. (2019); Papyan (2019;2020)]

SGD Happens in a Tiny Subspace?

Let us define the dominant subspace, bulk subspace & projection matrix $P_k(\Theta_t)$



(a) MLP on MNIST-5k

k largest eigenvalues $(\lambda_1, v_1), \ldots, (\lambda_k, v_k)$ of $\nabla^2 R(\Theta_t)$

$\rightarrow$ Dominant subspace at Θ_t: $\mathrm{span}(v_1, \ldots, v_k)$

$\rightarrow$ $P_k(\Theta_t)$: Projection matrix onto $\mathrm{span}(v_1, \ldots, v_k)$

The orthogonal complement of dominant subspace = bulk subspace

[Sagun et al. (2016;2017); Ghorbani et al. (2019); Papyan (2019;2020)]

SGD Happens in a Tiny Subspace?

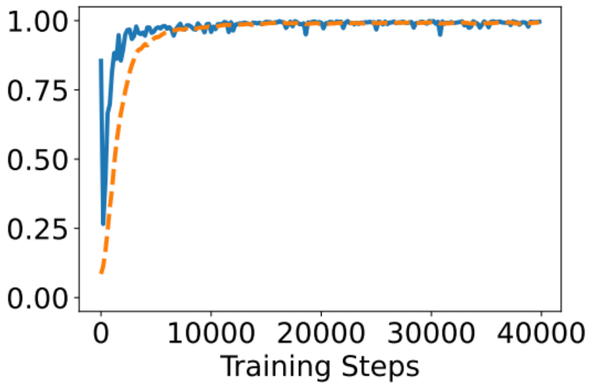
"SGD happens in the dominant subspace" [Gur-Ari et al. (2018)]

1. Train a NN with SGD → get a trajectory $\{\Theta_t\}_{t=0}^{T}$

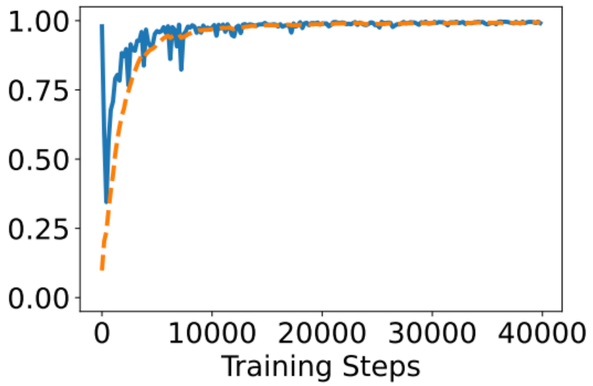
2. Calculate full-batch gradient $\nabla R(\Theta_t)$ and top-k dominant subspace of training loss evaluated at Θ_t

3. Measure alignment level: how much of $\nabla R(\Theta_t)$ aligns with the dominant subspace

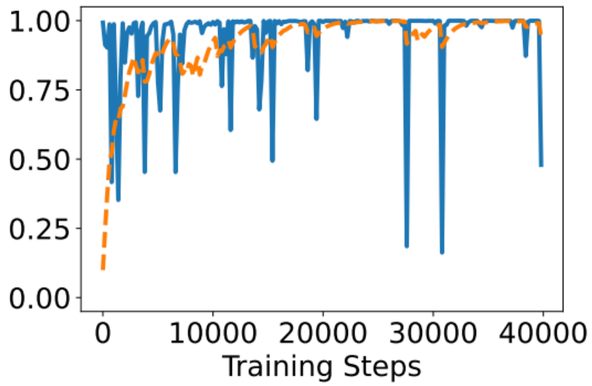
$$\frac{\|P_k(\Theta_t)\nabla R(\Theta_t)\|_2}{\|\nabla R(\Theta_t)\|_2}$$



(a) MLP on MNIST-5k



(b) CNN on CIFAR10-5k



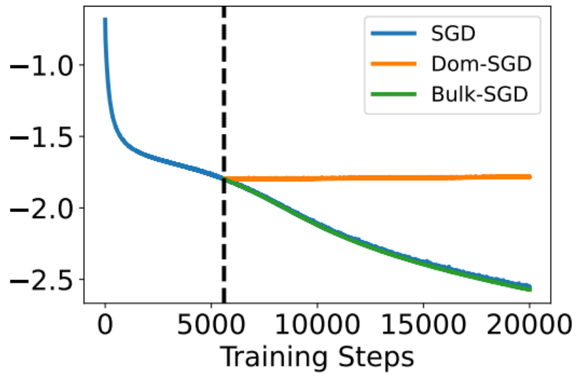
(c) Transformer on SST2-1k

Does SGD Really Work in Tiny Subspaces?

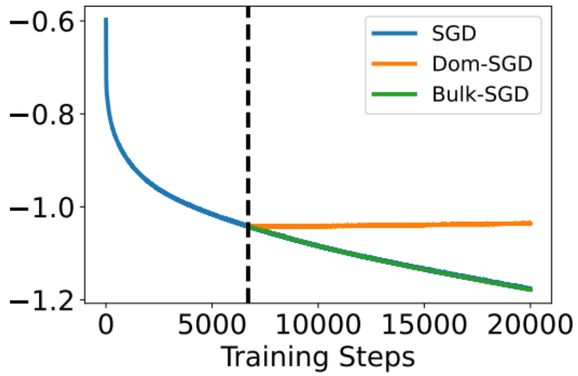
To test if this is really the case, we ran experiments using

$$\begin{cases} \Theta_{t+1} = \Theta_t - \eta g_t & \text{(SGD)} \\ \Theta_{t+1} = \Theta_t - \eta P_k(\Theta_t) g_t & \text{(Dom-SGD)} \\ \Theta_{t+1} = \Theta_t - \eta (I - P_k(\Theta_t)) g_t & \text{(Bulk-SGD)} \end{cases}$$

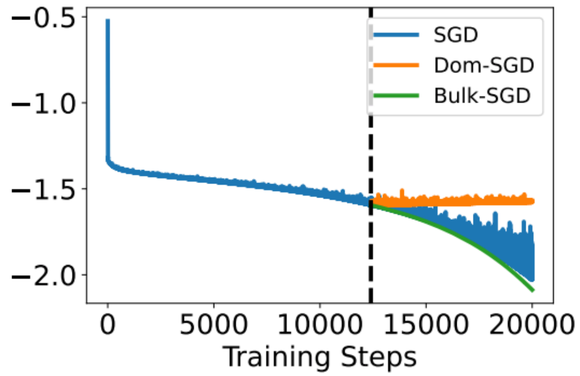
Surprisingly, Dom-SGD fails to decrease loss at all, while Bulk-SGD does, as if nothing changed! Loss descent happens due to the non-dominant components.



(a) MLP on MNIST-5k



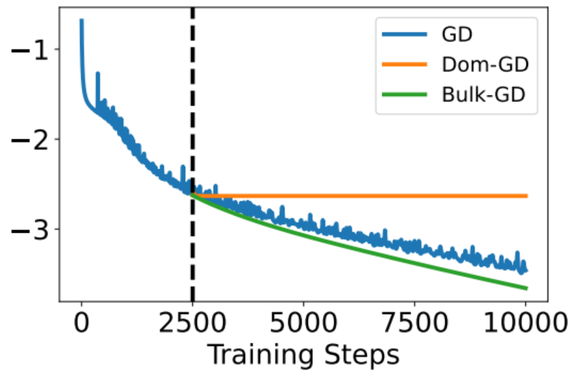
(b) CNN on CIFAR10-5k

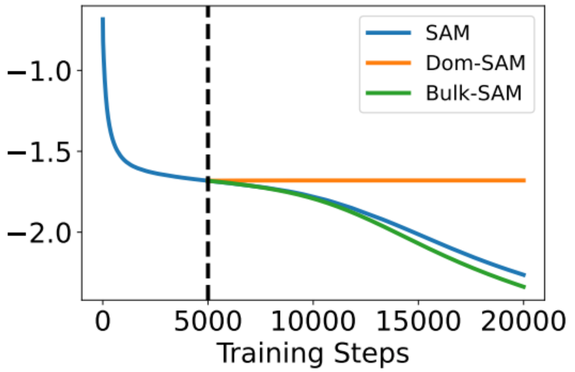


(c) Transformer on SST2-1k

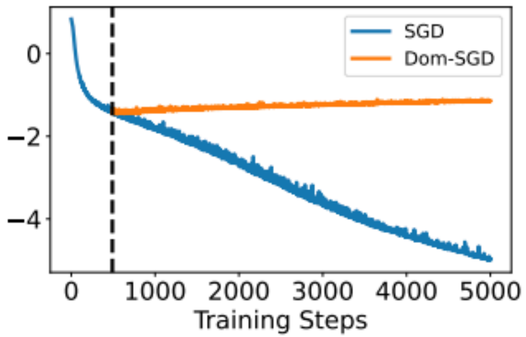
Does SGD Really Work in Tiny Subspaces?

The same observations continue to hold for GD/SAM in the EoS regime, and SGD with momentum & Adam, over a wide range of setups

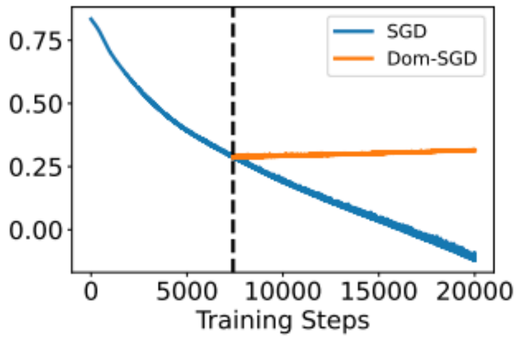




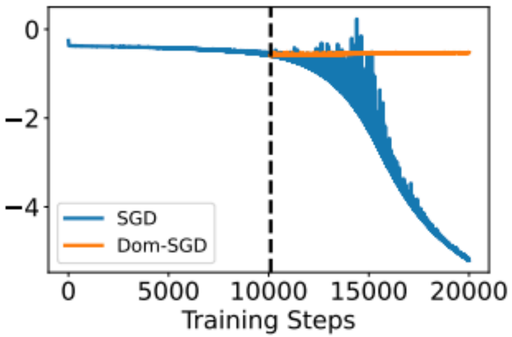
Consistent observation is made for cross-entropy loss: learning doesn't happen in the dominant eigenspace!



(a) MLP on MNIST-5k



(b) CNN on CIFAR10-5k



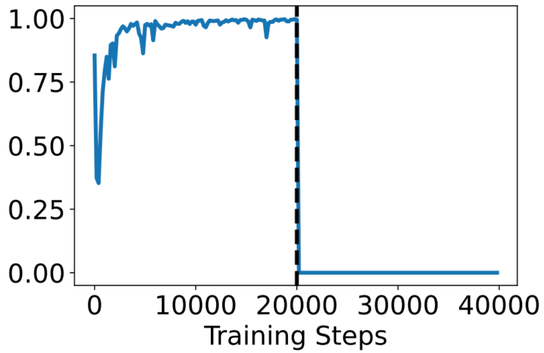
(c) Transformer on SST2-1k

[Song, Ahn, Y. (2025)]

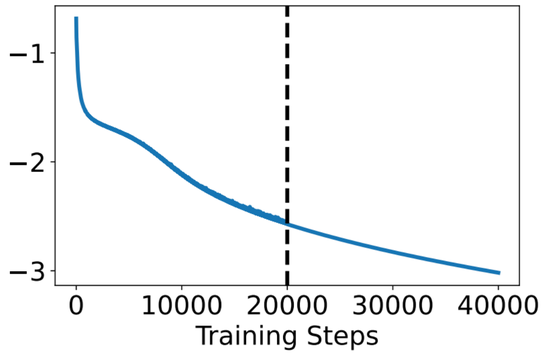
Change of Alignment due to Stochasticity

Other interesting facts: 1. GD (before EoS) doesn't show alignment
2. Yet, switching doesn't change loss curve that much

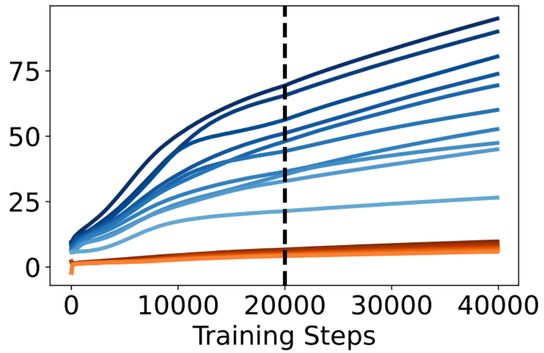
SGD → GD



(a) $\chi_{10}(\nabla L(\theta_t))$

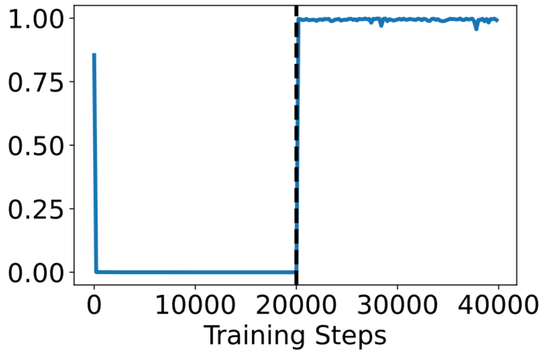


(b) Training loss (log-scale)

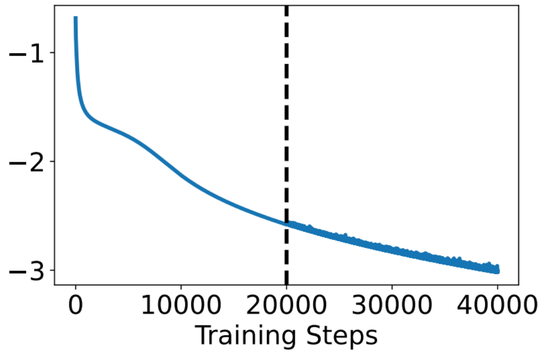


(c) Top-20 eigenvalues

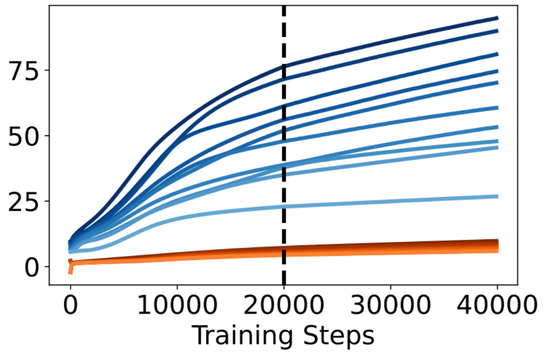
GD → SGD



(a) $\chi_{10}(\nabla L(\theta_t))$



(b) Training loss (log-scale)



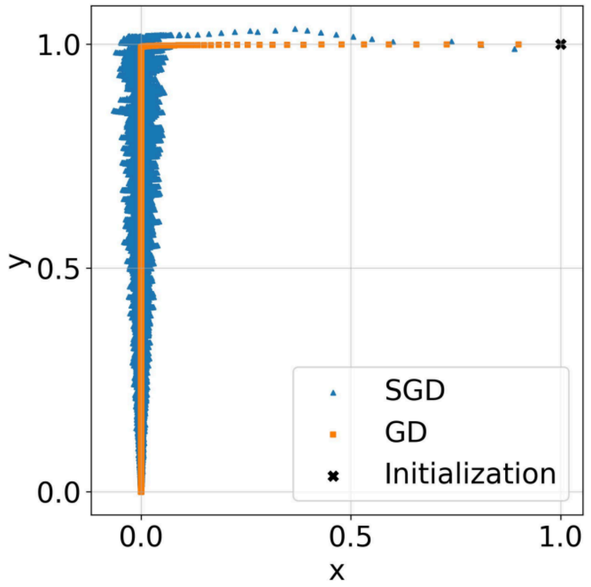
(c) Top -20 eigenvalues

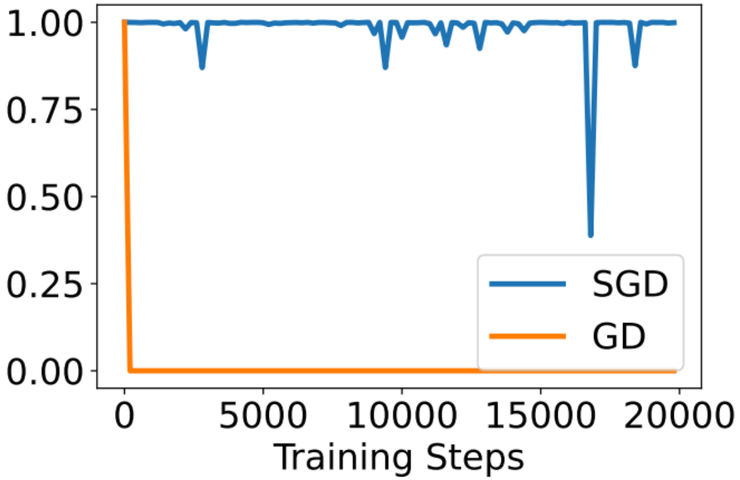
[Song, Ahn, Y. (2025)]

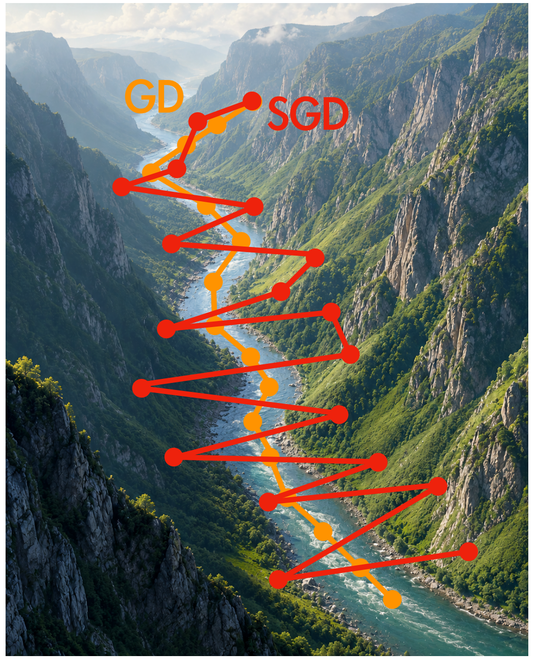
Explanation: River-Valley Landscape

Our understanding: learning happens in the bulk subspace,
but stochastic noise creates an illusion of alignment to dominant subspace

$$R(x, y) = \frac{1}{2}(1000x^2 + y^2)$$







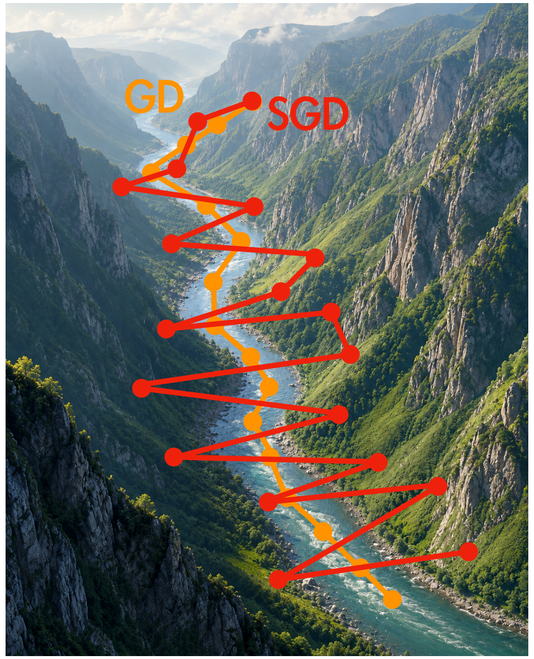
at $(x, y) = (0.01, 0.5)$, alignment ≈ 0.999

Explanation: River-Valley Landscape

Two qualitatively different types of directions:

- A few sharp directions dominate local curvature (largest eigenvalues of $\nabla^2 R$), but they are "learned" fast (Valley).
- Flat directions evolve much more slowly, but they drive long-term optimization progress (River).

→ Learning proceeds along a slowly moving flat "river," while the iterates rapidly move across the "valley."



[Song, Ahn, Y. (2025); Wen et al. (2025)]

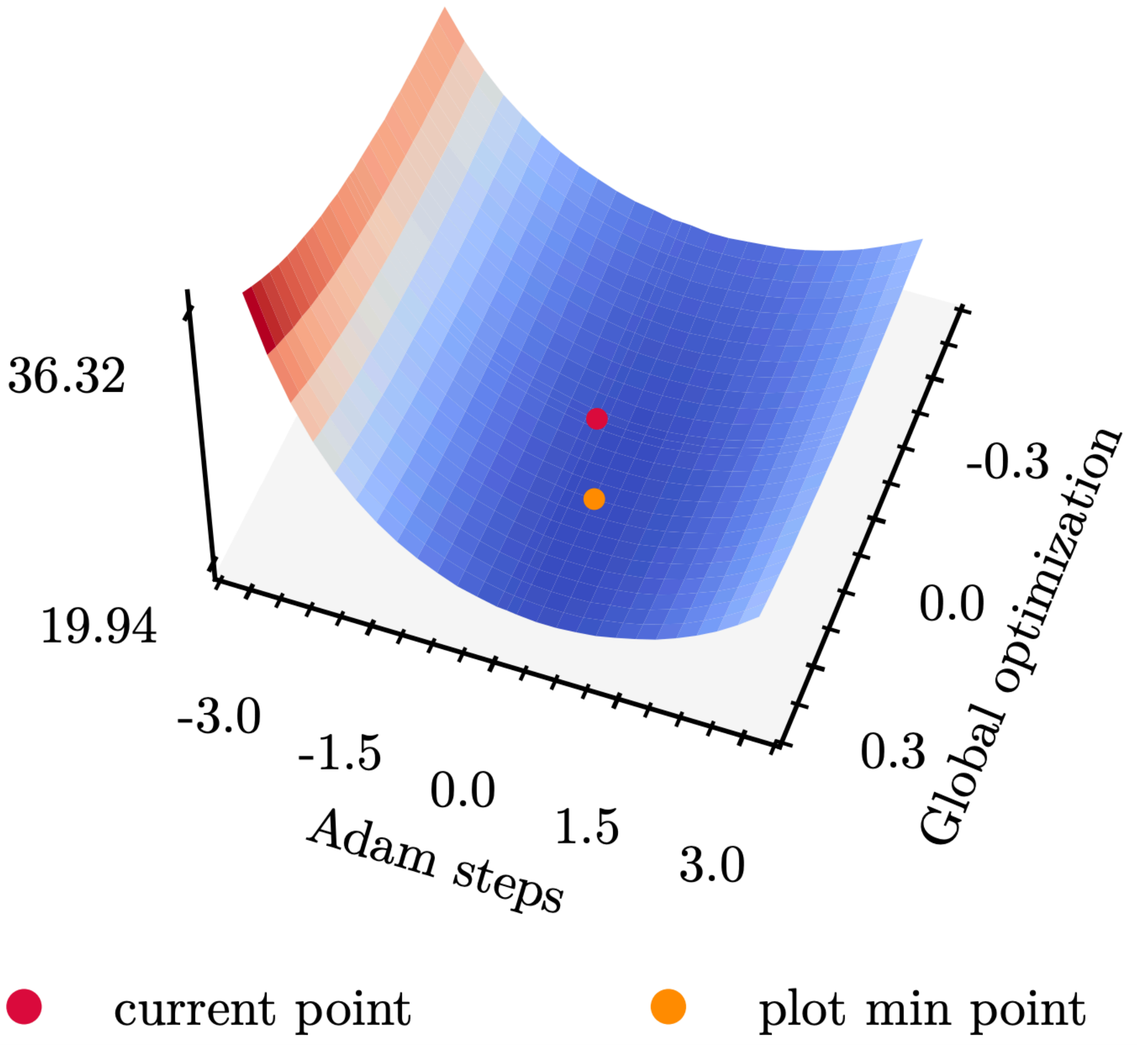
Explanation: River-Valley Landscape

Visualization in 210M Llama pretraining

Adam steps = $\Theta_{0.818T+10} - \Theta_{0.818T}$

Global opt = $\Theta_T - \Theta_{0.8T}$

(Axes not drawn to scale)



[Dremov et al. (2025)]

Some Historical Remarks

[Xing et al. (2018)]	SGD bounces between valley walls while staying above a valley floor. Focused on implication to SGD's flat minima.
[Andriushchenko et al. (2023)]	Large LR SGD jumps between valley walls, sustaining loss-driven sparsification. R-V identifies the river direction as key to optimization progress.
[Liu et al. (2022)]	Overparameterization can create curved manifolds of global minimizers. Visually similar geometry, but flat directions in R-V is key to learning progress.
[Song, Ahn, Y. (2025)]	Bulk subspace essential for training + spurious alignment to dominant subspace
[Wen et al. (2025)]	Understanding WSD LR schedule. Popularized the term "river-valley landscape"

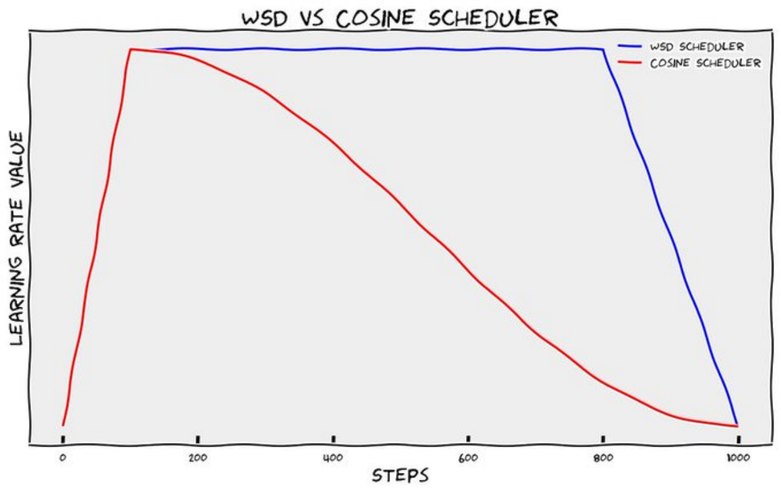
The definition of "river" varies by context + river subspace varies over training (even its dimension can)

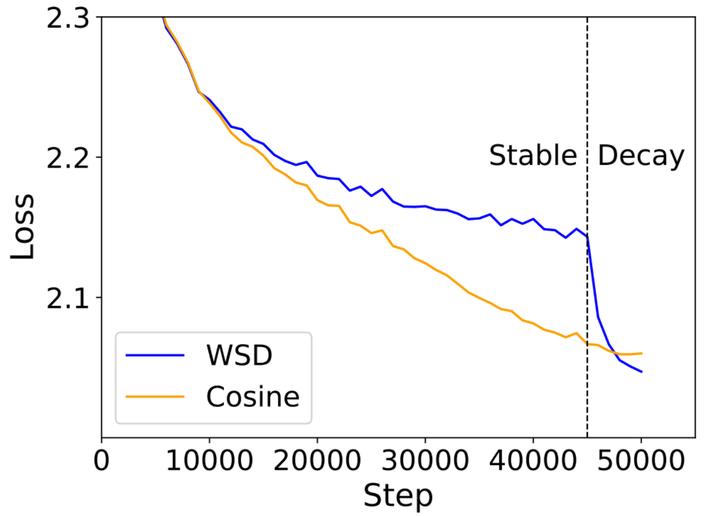
This talk: use the dominant (k-dim) vs bulk, direct evaluation only on toy models

River-Valley Landscape as a Useful Mental Model

River-valley landscape is useful in explaining many empirical phenomena...

- Fast decrease of training loss in when we decay step size





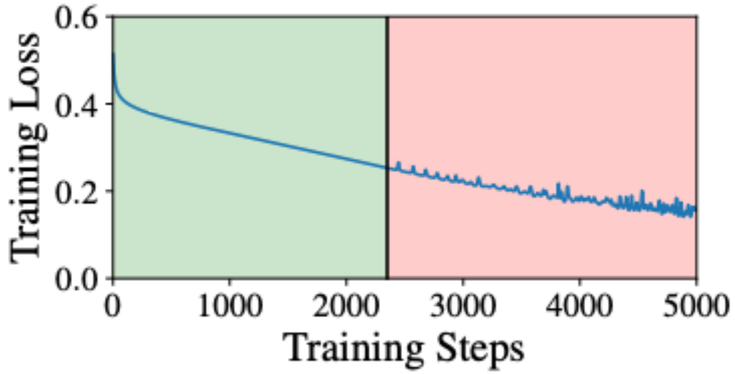


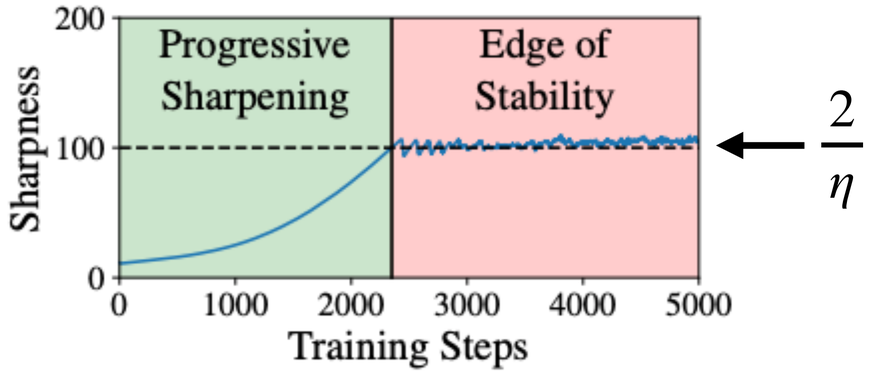
[Image from Elie Bakouch's X; Wen et al. (2025)]

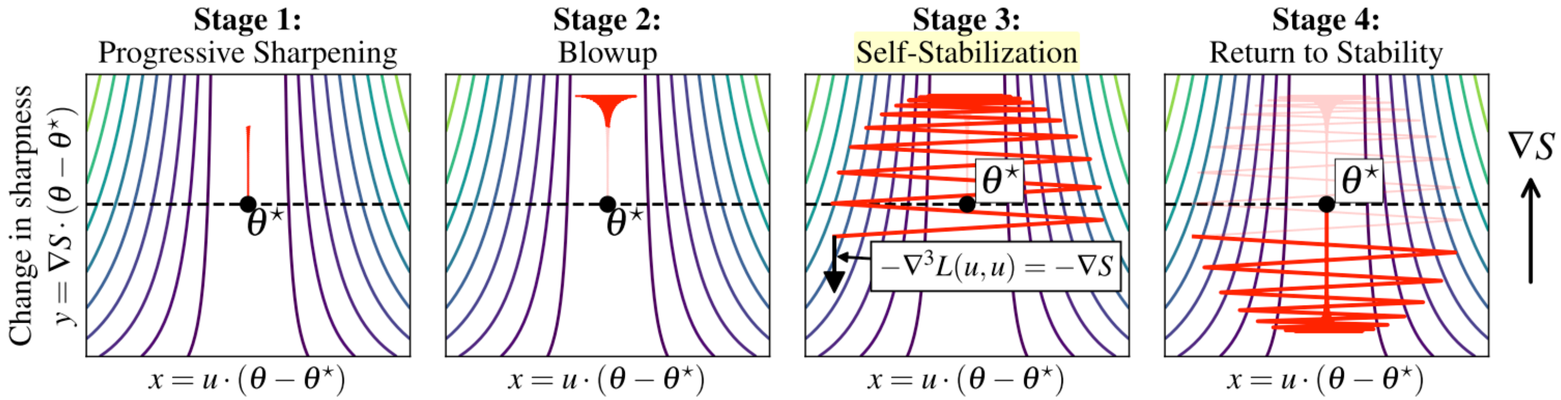
River-Valley Landscape as a Useful Mental Model

River-valley landscape is useful in explaining many empirical phenomena...

- Sharpness Dynamics in Edge of Stability regime







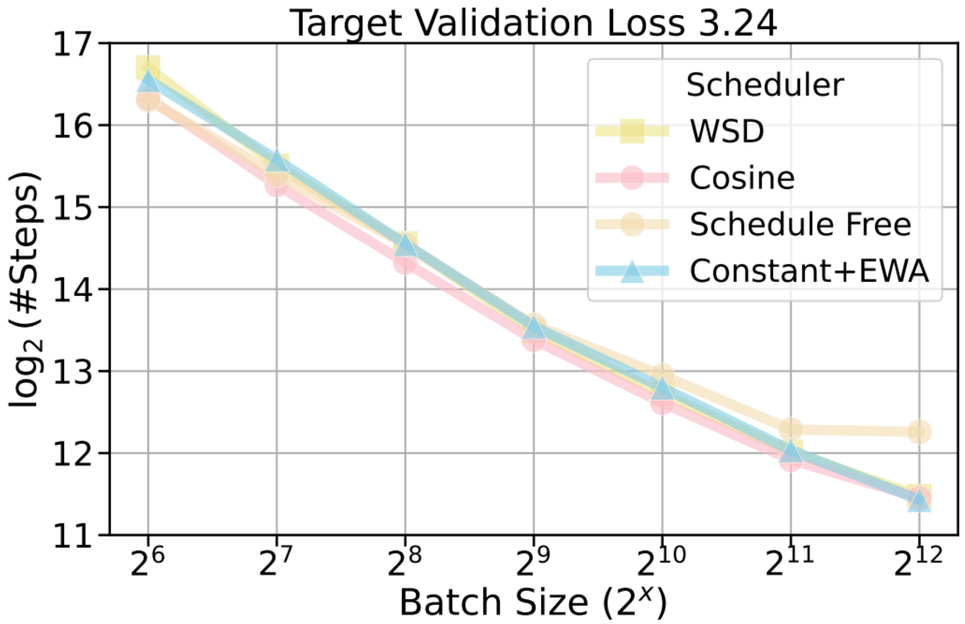


[Cohen et al. (2021); Damian et al. (2023)]

River-Valley Landscape as a Useful Mental Model

River-valley landscape is useful in explaining many empirical phenomena...

- Constant learning rate + Exponential Weight Averaging (EWA) helps





[Zhang et al. (2025)]

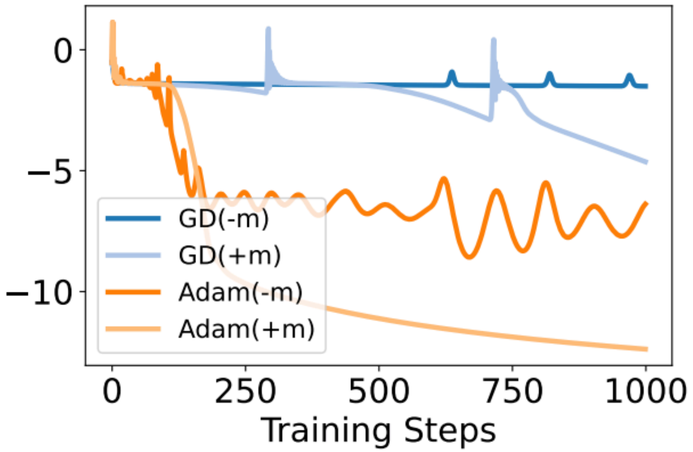
River-Valley Landscape as a Useful Mental Model

River-valley landscape is useful in explaining many empirical phenomena...

- Why momentum helps in neural network optimization

$$m_t \leftarrow \mu m_{t-1} + (1-\mu) g_t,\ \Theta_{t+1} \leftarrow \Theta_t - \eta m_t$$

Method	Mean Dom-LR	Mean Bulk-LR
GD(-m)	0.0100 (0.0000)	**0.0100** (0.0000)
GD(+m)	0.0070 (0.0232)	**0.0828** (0.0576)
Adam(-m)	0.0325 (0.0054)	**0.4672** (0.3555)
Adam(+m)	0.0004 (0.0101)	**2.6639** (1.2480)





Momentum makes Dom-LR ↓ Bulk-LR ↑

Faster convergence correlates with larger Bulk-LR

[Song, Ahn, Y. (2025)]

River-Valley Landscape as a Useful Mental Model

Optimizer dynamics can be "understood" via the river-valley landscape:

1. Superior performance of Schedule-free methods in neural network training

2. Superior performance of Muon in neural network training

   → AMUSE: how to combine their benefits

Schedule-Free Methods

Schedule-free optimizers [Defazio et al. (2024)]

$$y_t = (1-\beta)z_t + \beta x_t$$
$$z_{t+1} = z_t - \gamma \Delta_t$$
$$x_{t+1} = (1-\frac{1}{t+1})x_t + \frac{1}{t+1} z_{t+1} = \frac{1}{t+1}\sum_{i=1}^{t+1} z_i$$

where Δ_t is the base optimizer update based on gradient at y_t. We update z_t, while the algorithm returns x_T!

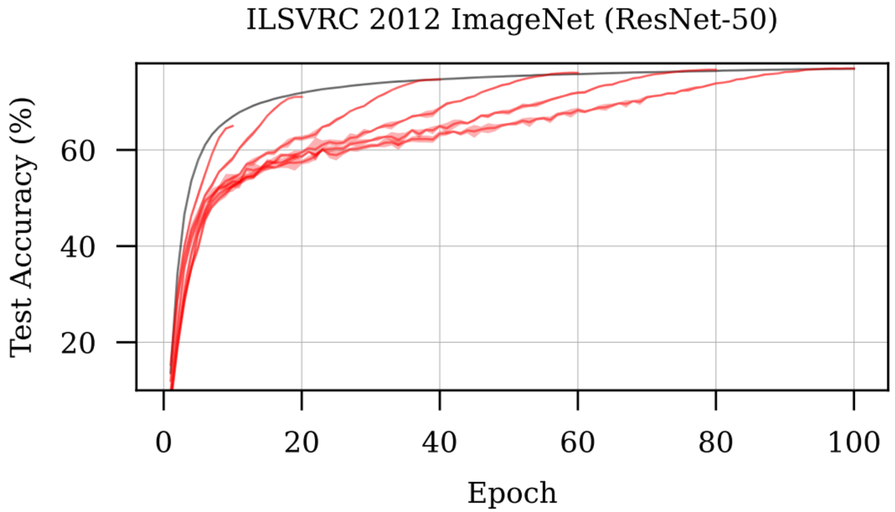
Algorithm 1 Schedule-Free AdamW

Input: x_1, learning rate γ, decay λ, warmup steps T_{warmup}, $\beta_1, \beta_2, \epsilon$
$z_1 = x_1$
$v_0 = 0$
for $t = 1$ **to** T **do**
  $y_t = (1-\beta_1)z_t + \beta_1 x_t$ ▷ Momentum via interpolation
  $g_t \in \partial f(y_t, \zeta_t)$ ▷ Gradient is evaluated at y
  $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$
  $\gamma_t = \gamma \sqrt{1-\beta_2^t}\min(1, t/T_{\text{warmup}})$ ▷ LR includes warmup and Adam bias-correction
  $z_{t+1} = z_t - \gamma_t g_t/(\sqrt{v_t}+\epsilon) - \gamma_t \lambda y_t$
  $c_{t+1} = \frac{\gamma_t^2}{\sum_{i=1}^t \gamma_i^2}$
  $x_{t+1} = (1-c_{t+1})\, x_t + c_{t+1} z_{t+1}$ ▷ Update weighted iterate average
end for
Return x_T

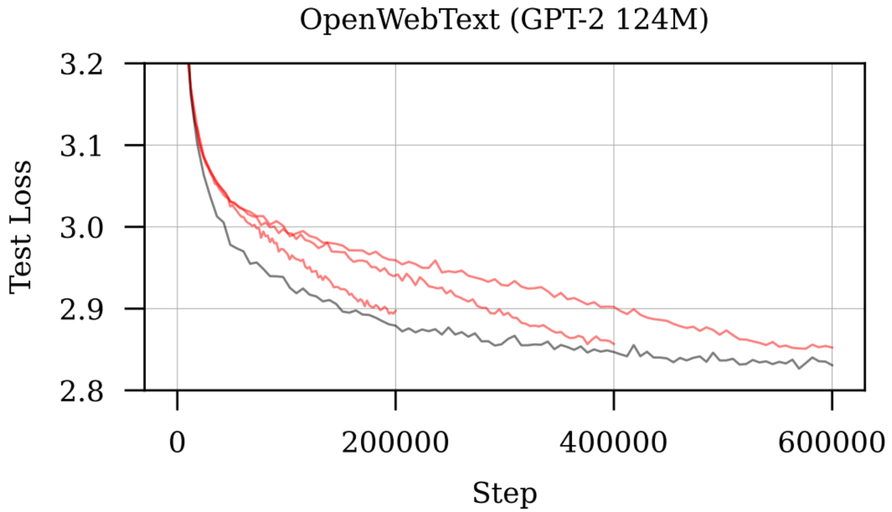
[Defazio et al. (2024)]

SF Methods Achieve Best of Scheduled Runs

In [Defazio et al. (2024)], Schedule-free methods were shown to achieve the Pareto frontier of performance vs time of LR-scheduled methods



Schedule-Free SGD vs LR-scheduled SGD



Schedule-Free AdamW vs LR-scheduled AdamW

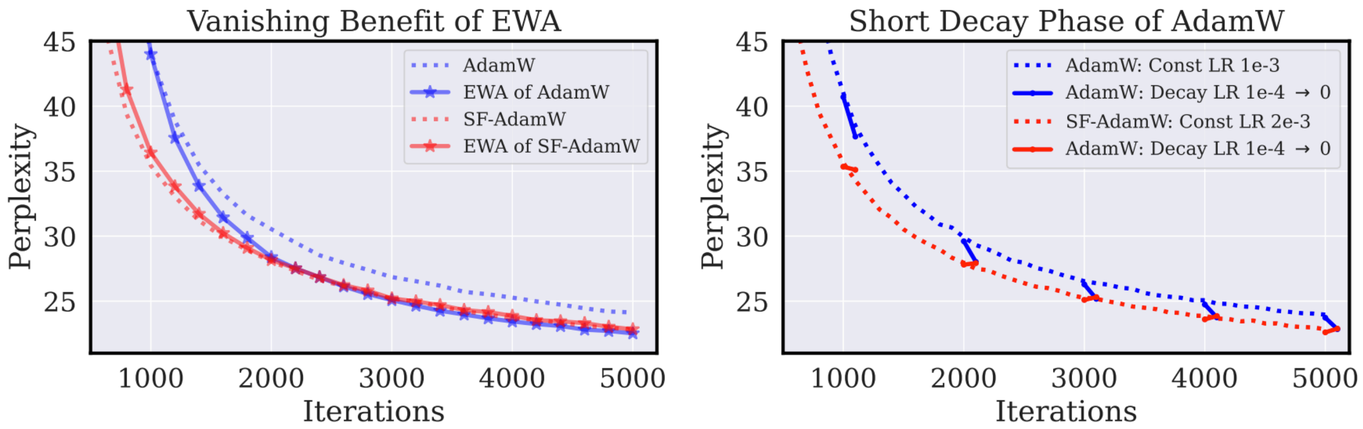
Schedule-free AdamW won the Self-Tuning track of MLCommons 2024 AlgoPerf Algorithmic Efficiency Challenge

[Defazio et al. (2024); https://mlcommons.org/benchmarks/algorithms/]

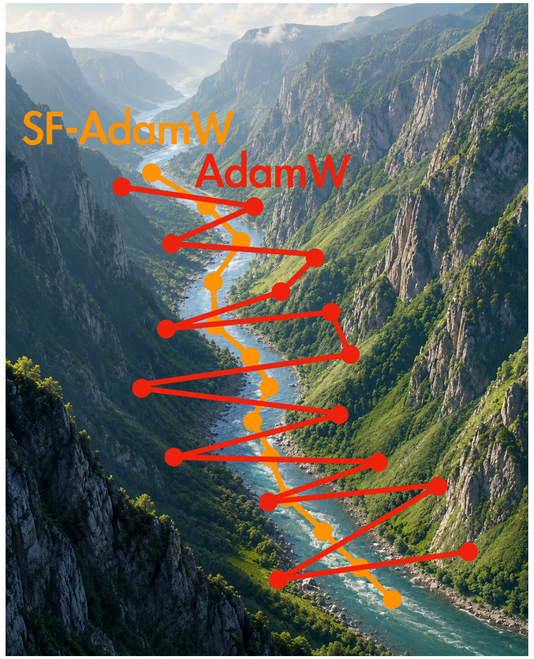
Understanding Schedule-Free AdamW

Why can Schedule-free AdamW achieve the Pareto frontier of scheduled runs?

Through LM pre-training experiments, we find that SF-AdamW with the right choice of β tracks the river closely.



Good choice $\beta = 0.95 \implies$ SF-AdamW follows the river

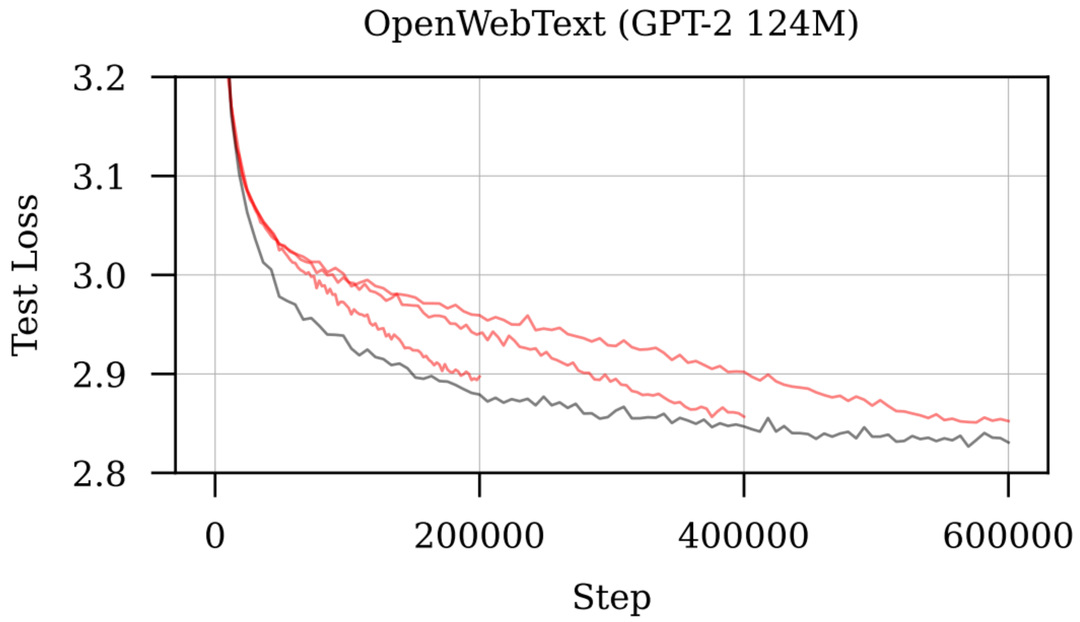


Understanding Schedule-Free AdamW

Why can Schedule-free AdamW achieve the Pareto frontier of scheduled runs?

AdamW with LR scheduling arrives near the river towards the end of training

SF-AdamW tracks the river throughout training
$\Longrightarrow$ achieves "best outcomes" along the way





[Song, Baek, Ahn, Y. (2025)]

Muon: Leveraging Matrix Structures in DL

Suppose we minimize a function $R(\boldsymbol{W})$ where $\boldsymbol{W} \in \mathbb{R}^{m \times n}$.

Traditional algorithms flatten $\boldsymbol{W}$ into a vector and make element-wise updates:

They do not use the matrix structure.

Gradient Descent (GD)

$$\boldsymbol{W}_{t+1} = \boldsymbol{W}_t - \eta \nabla R(\boldsymbol{W}_t)$$

$$|||$$

$$\mathrm{vec}(\boldsymbol{W}_{t+1}) = \mathrm{vec}(\boldsymbol{W}_t) - \eta \mathrm{vec}(\nabla R(\boldsymbol{W}_t))$$

Adam/AdamW

$$\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1)\mathrm{vec}(\nabla R(\boldsymbol{W}_t))$$

$$\boldsymbol{v}_t = \beta_2 \boldsymbol{v}_{t-1} + (1-\beta_2)\mathrm{vec}(\nabla R(\boldsymbol{W}_t))^{\odot 2}$$

$$\mathrm{vec}(\boldsymbol{W}_{t+1}) = \mathrm{vec}(\boldsymbol{W}_t) - \eta \boldsymbol{v}_t^{-1/2} \odot \boldsymbol{m}_t \; (- \eta \lambda \mathrm{vec}(\boldsymbol{W}_t))$$

[Kingma & Ba (2015); Loshchilov & Hutter (2019)]

Muon: Leveraging Matrix Structures in DL

New optimizers: Use matrix structure of parameters!

GD

$$\boldsymbol{W}_{t+1} = \boldsymbol{W}_t - \eta \nabla R(\boldsymbol{W}_t)$$

Spectral GD

$$\boldsymbol{W}_{t+1} = \boldsymbol{W}_t - \eta \mathcal{T}(\nabla R(\boldsymbol{W}_t))$$

where $\mathcal{T}$ is the orthogonalization operator.

(if SVD $\boldsymbol{A} = \boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^T$, then $\mathcal{T}(\boldsymbol{A}) = \boldsymbol{U}\boldsymbol{V}^T$)

[Carlson et al. (2015)]

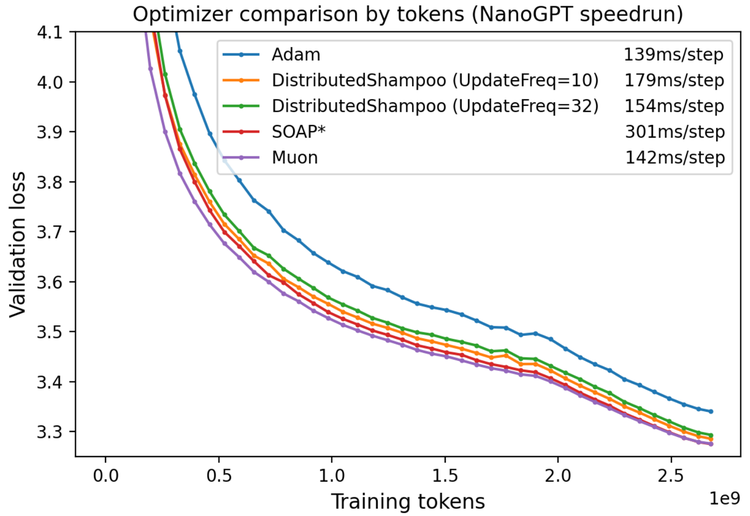
Muon: Leveraging Matrix Structures in DL

Muon: SpecGD, but

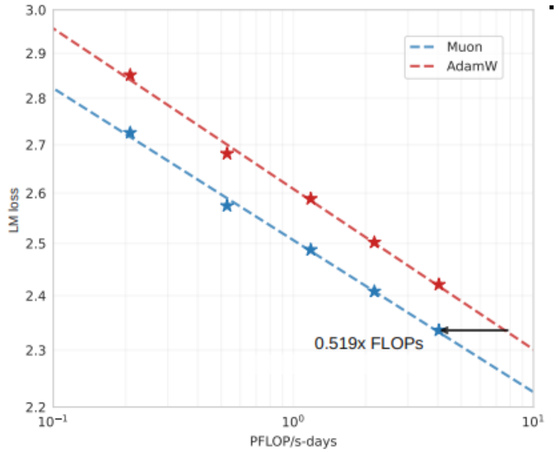
- add Polyak/Nesterov momentum
- approximate but efficient version of $\mathcal{T}$ by Newton-Schulz iterations

$$\boldsymbol{M}_t = \mu \boldsymbol{M}_{t-1} + (1-\mu)\nabla R(\boldsymbol{W}_t)$$

$$\boldsymbol{W}_{t+1} = \boldsymbol{W}_t - \eta \hat{\mathcal{T}}(\mu \boldsymbol{M}_t + (1-\mu)\nabla R(\boldsymbol{W}_t))$$



MUON IS SCALABLE FOR LLM TRAINING



[Jordan et al. (2024); Kimi Team (2025)]

Understanding Muon via River Valley Lens

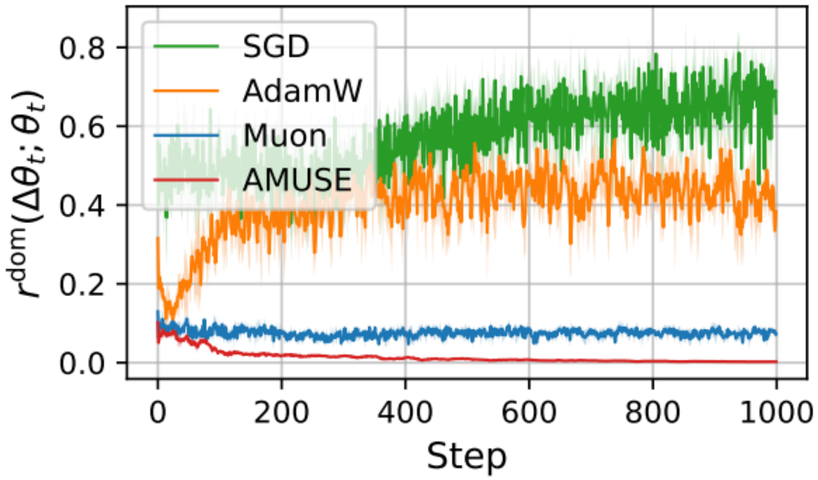
Muon has relatively larger river components (bulk subspace) in its updates. Why?

Muon's orthogonalization makes singular values uniform

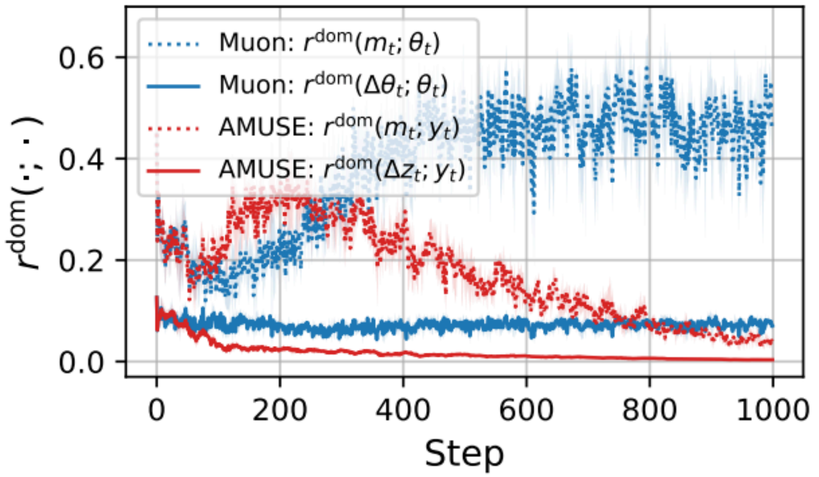
$\Longrightarrow$ river (bulk) update directions are relatively amplified

$\Longrightarrow$ faster optimization progress

$$r^{dom}(\boldsymbol{v};\Theta) := \frac{\|P_k(\Theta)\boldsymbol{v}\|_2}{\|\boldsymbol{v}\|_2}$$



(a) Muon vs. SGD/AdamW



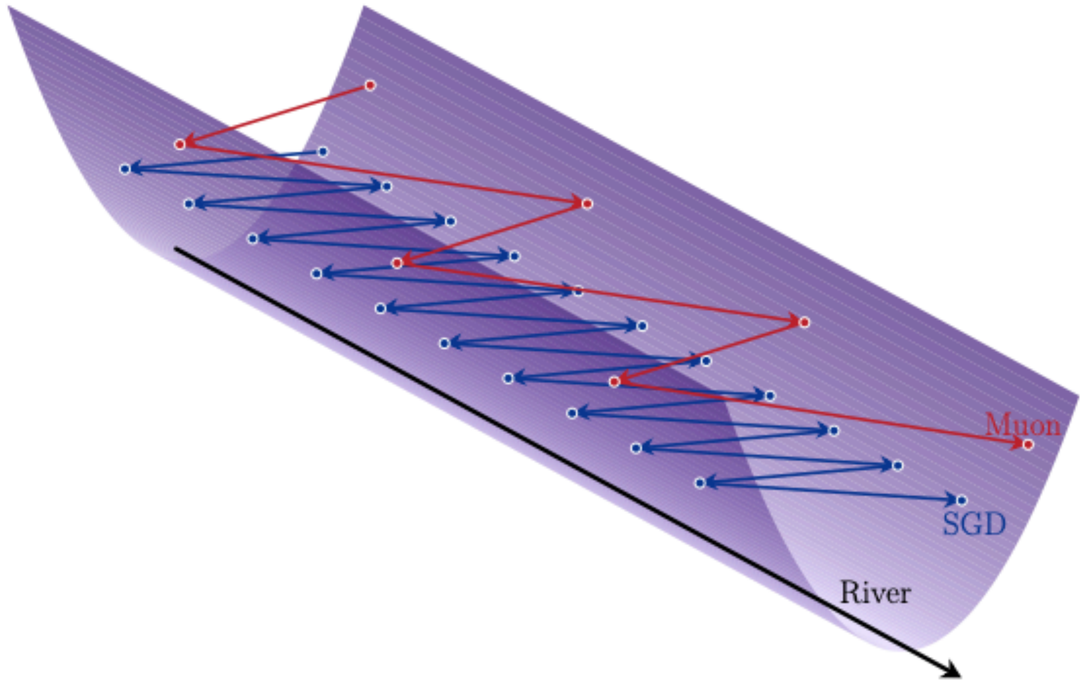
(b) Momentum $\boldsymbol{m}_t$ vs. Update $\Delta\theta_t\,(=\mathcal{O}(\boldsymbol{m}_t))$

Understanding Muon via River Valley Lens

Muon is a normalized method, so the valley components don't decay to zero
$\Longrightarrow$ oscillatory along the valley directions and requires learning rate scheduling

We saw Schedule-free methods can make iterates run through the river...

How about we combine SF + Muon?

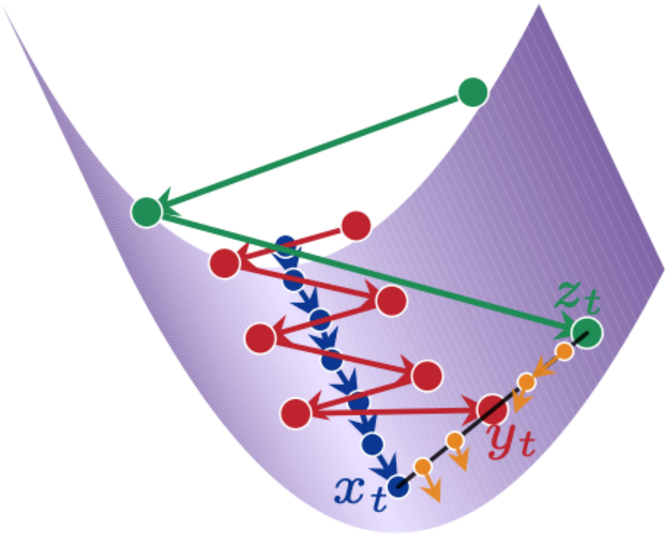


[Kim, Shin, Yun, Baek, Song, Y. (2026)]

AMUSE: Anytime Muon with Stable Gradient Evaluation

We propose a new algorithm AMUSE:

$$\boldsymbol{Y}_t = (1-\beta_t)\boldsymbol{Z}_t + \beta_t \boldsymbol{X}_t$$
$$\boldsymbol{M}_t = \mu \boldsymbol{M}_{t-1} + (1-\mu)\nabla R(\boldsymbol{Y}_t)$$
$$\boldsymbol{Z}_{t+1} = \boldsymbol{Z}_t - \eta \hat{\mathcal{T}}(\mu \boldsymbol{M}_t + (1-\mu)\nabla R(\boldsymbol{Y}_t))$$
$$\boldsymbol{X}_{t+1} = (1-c_{t+1})\boldsymbol{X}_t + c_{t+1}\boldsymbol{Z}_{t+1}$$



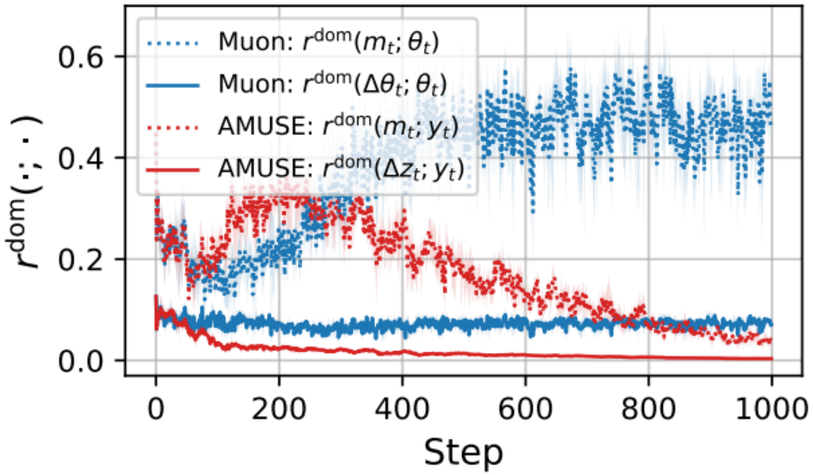
This is "Schedule-free Muon" except:

- We retain momentum in Muon (cf. SF-AdamW discarded momentum in AdamW)
- We schedule β_t so that it starts at a smaller value and increases to 1
  $\Longrightarrow$ gradient evaluated closer to the river $\Longrightarrow$ larger river component in $\boldsymbol{M}_t$

AMUSE: Anytime Muon with Stable Gradient Evaluation

We propose a new algorithm AMUSE:

$$
\begin{aligned}
\boldsymbol{Y}_t &= (1-\beta_t)\boldsymbol{Z}_t + \beta_t \boldsymbol{X}_t \\
\boldsymbol{M}_t &= \mu \boldsymbol{M}_{t-1} + (1-\mu)\nabla R(\boldsymbol{Y}_t) \\
\boldsymbol{Z}_{t+1} &= \boldsymbol{Z}_t - \eta \hat{\mathcal{T}}(\mu \boldsymbol{M}_t + (1-\mu)\nabla R(\boldsymbol{Y}_t)) \\
\boldsymbol{X}_{t+1} &= (1-c_{t+1})\boldsymbol{X}_t + c_{t+1}\boldsymbol{Z}_{t+1}
\end{aligned}
$$



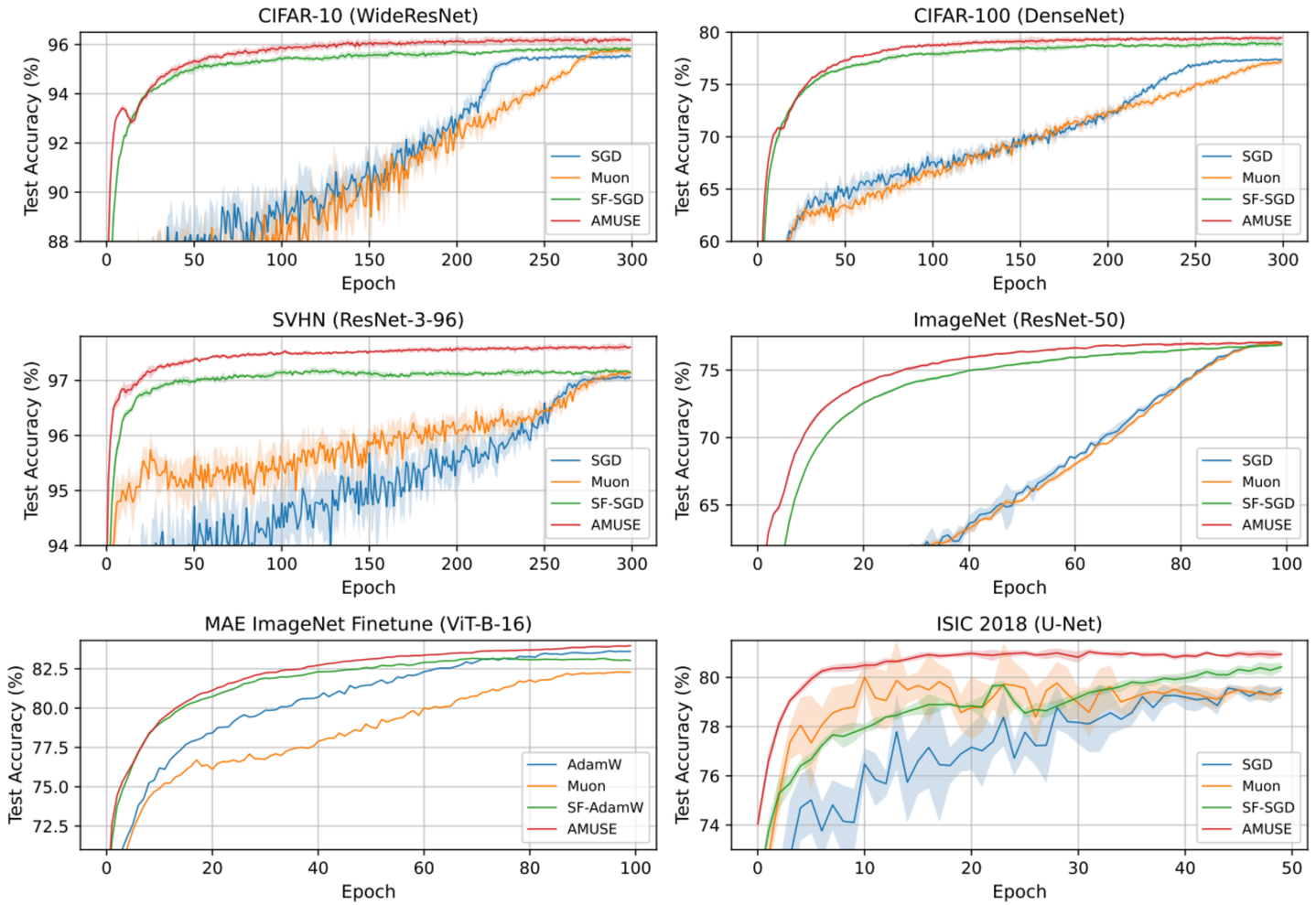
(b) Momentum $\boldsymbol{m}_t$ vs. Update $\Delta\theta_t\ (= \mathcal{O}(\boldsymbol{m}_t))$

This is "Schedule-free Muon" except:

- We retain momentum in Muon (cf. SF-AdamW discarded momentum in AdamW)
- We schedule β_t so that it starts at a smaller value and increases to 1
  $\Longrightarrow$ gradient evaluated closer to the river $\Longrightarrow$ larger river component in $\boldsymbol{M}_t$

AMUSE: Anytime Muon with Stable Gradient Evaluation

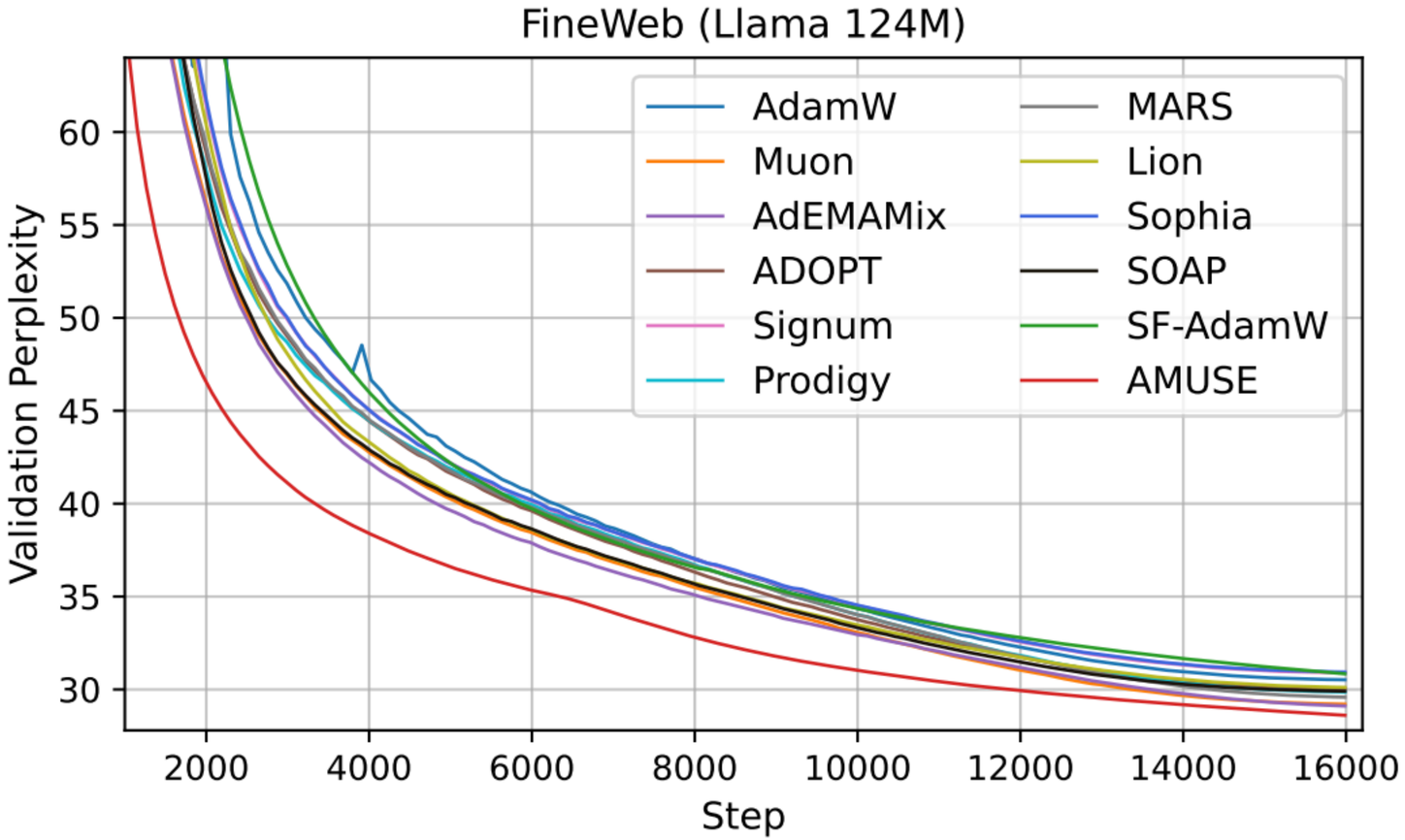
Experiments on image tasks



[Kim, Shin, Yun, Baek, Song, **Y.** (2026)]

AMUSE: Anytime Muon with Stable Gradient Evaluation

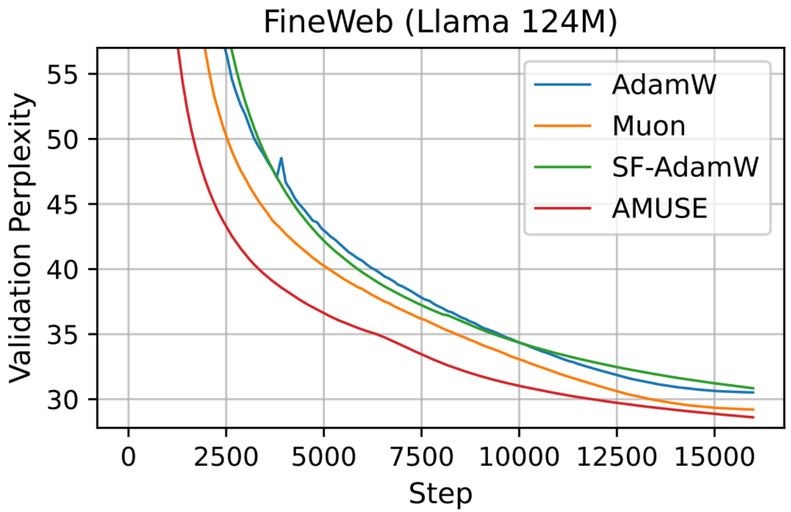
Experiments on LLM pretraining

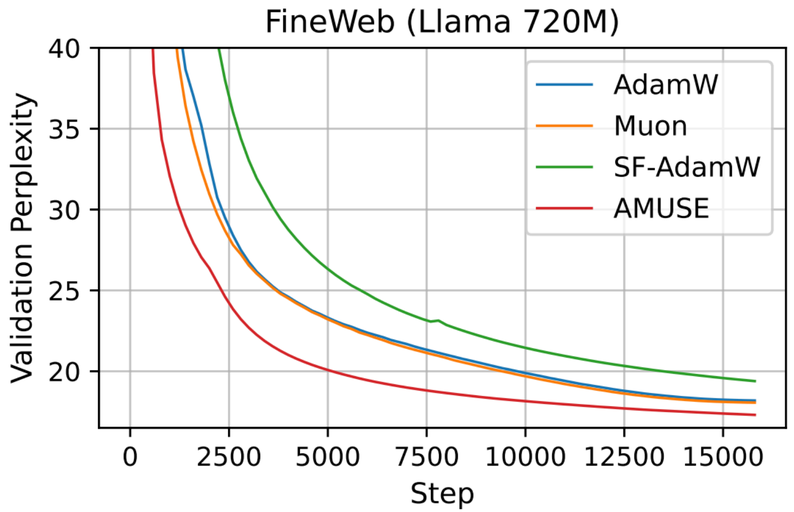


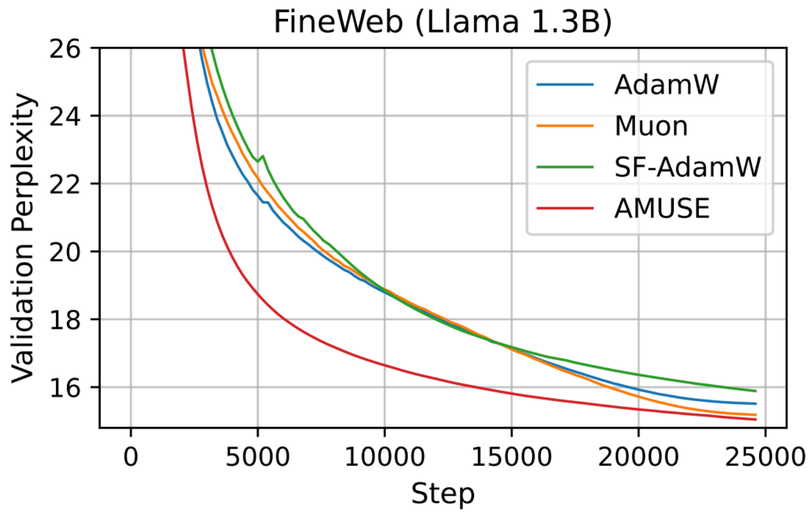
[Semenov et al. (2025); Kim, Shin, Yun, Baek, Song, **Y.** (2026)]

AMUSE: Anytime Muon with Stable Gradient Evaluation

Experiments on LLM pretraining



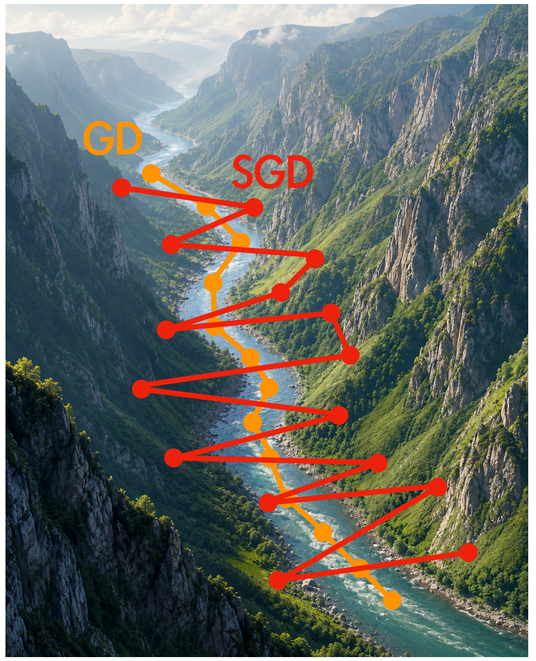




[Kim, Shin, Yun, Baek, Song, **Y.** (2026)]

Summary

- NN training doesn't happen in dominant subspace + spurious alignment arises
- River-valley loss landscape hypothesis
  - Helps understand NN phenomena (LR scheduling, EoS, EWA, momentum...)
- Understanding SF methods and Muon via river-valley lens
- Designing a new optimizer AMUSE from the intuitions



Thank you! Questions?

Xing et al. A Walk with SGD. 2018

Liu et al. Loss landscapes and optimization in over-parameterized non-linear systems and neural networks. 2022

Andriushchenko et al. SGD with Large Step Sizes Learns Sparse Features. 2023

Song, Ahn, Yun. Does SGD really happen in tiny subspaces? 2025

Wen et al. Understanding Warmup-Stable-Decay Learning Rates: A River Valley Loss Landscape Perspective. 2025

Defazio et al. The Road Less Scheduled. 2024

Song, Baek, Ahn, Yun. Through the River: Understanding the Benefit of Schedule-Free Methods for Language Model Training. 2025

Jordan et al. Muon: An optimizer for hidden layers in neural networks. 2024

Kim, Shin, Yun, Baek, Song, Yun. AMUSE: Anytime Muon with Stable Gradient Evaluation. 2026

Chulhee Yun (chulhee.yun@kaist.ac.kr)

OptiML Optimization & Machine Learning Laboratory

KAIST AI

Kim Jaechul Graduate School